

Intel MKTME AES-XTS-256 コード変更チートシート

現在の working tree から抽出した変更後コード。 `git diff` と変更前 snapshot の差分を照合済み。

コードは変更箇所の抜粋（ `program_test_key()` のみ関数全体）。抜粋を関数全体と置き換えないこと。

1. arch/x86/include/asm/intel_pconfig.h

変更コード

```
/* ENC_ALG is a bitmap: AES-XTS-256 is bit 2, not policy encoding 2.
 */
#define MKTME_AES_XTS_256      (1 << 10)
```

説明

`KEYID_CTRL[23:8]` の `ENC_ALG` は bitmap。256用の bit 2 を 8bit シフトし、
`1 << 10 = 0x400` とする。

KeyID0 policy の数値 `2` とは別物。既存の `MKTME_AES_XTS_128` 定義は変更していない。

2. arch/x86/kernel/cpu/intel.c

変更コード

```
#define TME_ACTIVATE_POLICY_AES_XTS_256 2
```

```
#define TME_ACTIVATE_CRYPT0_AES_XTS_256 (1 << 2)
```

```
tme_policy = TME_ACTIVATE_POLICY(tme_activate);
if (tme_policy != TME_ACTIVATE_POLICY_AES_XTS_256) {
    pr_err("x86/tme: AES-XTS-256 policy required: policy=%#llx
```

```

activate=%#llx\n",
        tme_policy, tme_activate);
mktme_status = MKTME_DISABLED;
}

tme_crypto_algs = TME_ACTIVATE_CRYPT0_ALGS(tme_activate);
if (!(tme_crypto_algs & TME_ACTIVATE_CRYPT0_AES_XTS_256)) {
    pr_err("x86/mktme: AES-XTS-256 not activated: algs=%#llx\n",
        tme_crypto_algs, tme_activate);
    mktme_status = MKTME_DISABLED;
}

```

説明

`detect_tme()` が読む `IA32_TME_ACTIVATE (0x982)` の `[7:4]` は policy `2`、`[63:48]` は bit 2 (raw bit 50) を要求する。

どちらかを満たさなければ `mktme_status = MKTME_DISABLED`。128専用判定を256専用判定に変更した。

3. arch/x86/mm/mktme_test.c

test_configure()

変更コード

```

/* Intel memory encryption spec 336907-004US, Tables 4-1/4-2. */
#define TEST_TME_CAP_AES_XTS_256 BIT_ULL(2)
#define TEST_TME_ACT_AES_XTS_256 BIT_ULL(50)
#define TEST_TME_POLICY_AES_XTS_256 2

```

```

cfg->bits = (cfg->activate >> 32) & 15;
width = cpuid_eax(0x80000008) & 255;
if (!(cfg->activate & BIT_ULL(0))) {
    pr_err("TME activation is not locked\n");
}

```

```
        return -EOPNOTSUPP;
    }
    if (!(cfg->activate & BIT_ULL(1))) {
        pr_err("TME encryption hardware is not enabled\n");
        return -EOPNOTSUPP;
    }
    if (cfg->activate & BIT_ULL(31)) {
        pr_err("KeyID0 TME encryption is bypassed\n");
        return -EOPNOTSUPP;
    }
    if (((cfg->activate >> 4) & 15) != TEST_TME_POLICY_AES_XTS_256) {
        pr_err("KeyID0 policy is not AES-XTS-256: policy=%#llx\n",
              (cfg->activate >> 4) & 15);
        return -EOPNOTSUPP;
    }
    if (!(cfg->activate & TEST_TME_ACT_AES_XTS_256)) {
        pr_err("AES-XTS-256 is not activated for MKTME\n");
        return -EOPNOTSUPP;
    }
    if (!(cfg->capability & TEST_TME_CAP_AES_XTS_256)) {
        pr_err("AES-XTS-256 capability missing\n");
        return -EOPNOTSUPP;
    }
    if (cfg->bits < 2 || cfg->bits > ((cfg->capability >> 32) & 15))
    {
        pr_err("invalid KeyID bits: active=%u capability_max=%llu\n",
              cfg->bits, (cfg->capability >> 32) & 15);
        return -EOPNOTSUPP;
    }
}
```

説明

capability は `0x981[2]`、MKTME activation は `0x982[50]`、KeyID0 policy は `[7:4] == 2` に変更。

lock・enable・bypass・KeyID bits の条件は維持し、拒否理由を個別表示する。

`cfg->bits` は既存の `[35:32]` 抽出のまま。提示MSRでは有効4bit／最大4bit。

program_test_key()

変更コード

```
static int program_test_key(struct mktme_key_program *prog,
                           unsigned int keyid, bool clear)
{
    int status;

    if (keyid != 1 && keyid != 2)
        return -EINVAL;
    /* RANDOM uses CPU entropy; keep all reserved bytes (upper 32B/
    key) zero. */
    memset(prog, 0, sizeof(*prog));
    prog->keyid = keyid;
    /* RFC #23: even CLEAR_KEY needs an activated algorithm. */
    prog->keyid_ctrl = MKTME_AES_XTS_256 |
        (clear ? MKTME_KEYID_CLEAR_KEY : MKTME_KEYID_SET_KEY_RANDOM);
    status = mktme_key_program(prog);
    if (status != MKTME_PROG_SUCCESS)
        pr_err("Key%u %s PCONFIG ctrl=%#x status=%d\n", keyid,
              clear ? "clear" : "program", prog->keyid_ctrl,
              status);
    /* Bounded experiment: no hidden retry or partial-success
    acceptance. */
    return status == MKTME_PROG_SUCCESS ? 0 : -EIO;
}
```

説明

RANDOM／CLEAR の両方に **MKTME_AES_XTS_256** を指定し、CTRL はそれぞれ **0x401**／**0x402**。

全体ゼロ初期化とKeyID 1/2制限は維持。PCONFIG失敗ログに生成したCTRLを追加した。

診断追加（重要箇所のみ）

変更コード — test_configure(): raw MSR／起動時disabled

```

/* Log raw MSRs even when detect_tme() disabled the experiment.
*/
if (rdmsrl_safe(TEST_TME_ACTIVATE, &cfg->activate)) {
    pr_err("cannot read IA32_TME_ACTIVATE (0x982)\n");
    return -EIO;
}
pr_info("IA32_TME_ACTIVATE (0x982)=%#018llx\n", cfg->activate);
if (rdmsrl_safe(TEST_TME_CAPABILITY, &cfg->capability)) {
    pr_err("cannot read IA32_TME_CAPABILITY (0x981)\n");
    return -EIO;
}
pr_info("IA32_TME_CAPABILITY (0x981)=%#018llx\n", cfg->capability);
if (!intel_mktme_test_enabled()) {
    pr_err("detect_tme() disabled MKTME; see x86/tme and x86/
mktme boot logs\n");
    return -EOPNOTSUPP;
}

```

説明

起動時のMKTME状態を判定する前に **0x982** / **0x981** を表示し、disabledとMSR読取失敗を区別する。

変更コード — test_check_cpu(): CPU型番 / Directory Mode / MSR不一致

```

if (c->x86_vendor != X86_VENDOR_INTEL || c->x86 != 6 ||
    (c->x86_model != 0x6a && c->x86_model != 0x6c)) {
    pr_err("CPU%d: unaudited CPU vendor=%u family=%#x
model=%#x\n",
           smp_processor_id(), c->x86_vendor, c->x86, c->x86_model);
    goto bad;
}

```

```

if (status & BIT_ULL(8)) {

```

```

        pr_err("CPU%d: Directory Mode prohibits aliases:
MSR[0x6f]=%#018llx\n",
                smp_processor_id(), status);
        goto bad;
    }
    if (act != cfg->activate || cap != cfg->capability) {
        pr_err("CPU%d: inconsistent MSRs: activate=%#018llx
expected=%#018llx capability=%#018llx expected=%#018llx\n",
                smp_processor_id(), act, cfg->activate, cap, cfg-
>capability);
        goto bad;
    }

```

説明

既存のCPU型番制約、Directory Mode禁止、CPU間MSR一致条件に診断を追加。**bad** 経由で拒否する構造は維持。

変更コード — test_open_session(): EFI属性

```

    if (!test_page_encryptable(s->page)) {
        pr_err("payload PFN=%#lx lacks EFI CPU_CRYPT0+WB coverage
(saved_ranges=%u)\n",
                page_to_pfn(s->page), nr_crypto_ranges);
        ret = -EOPNOTSUPP;
        goto fail;
    }

```

説明

既存のEFI **CPU_CRYPT0+WB** 条件で拒否されたとき、割当PFNと保存領域数を表示する。

4. AES-XTS-256 encodingまとめ

項目	AES-XTS-256値
----	--------------

項目	AES-XTS-256値
IA32_TME_CAPABILITY	bit 2
IA32_TME_ACTIVATE MKTME algorithm	bit 50
KeyID0 policy [7:4]	2
PCONFIG ENC_ALG bitmap	4
KEYID_CTRL algorithm bits	0x400
RANDOM	0x401
CLEAR	0x402

現在の定義・CTRL生成式と、[Intel仕様 rev.1.4](#)（[Tables 4-1/4-2](#)、[§6.2.1.1](#)）に一致する。
 提示値 `0x981=0x000000F440000004`／`0x982=0x0004000400000027` は capability bit2、activation bit50、policy=2 を満たす。

5. 変更していない部分

128→256変更では、`physical_mask`、`PTE_PFN_MASK`、`PTE_PFN_MASK_MAX`、PFN/KeyID separation、alias mappingの基本構造、使用するKeyID 1／KeyID 2は変更していない。